JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Integer

java.lang.Object
    java.lang.Number
        java.lang.Integer

All Implemented Interfaces:

Serializable, Comparable<Integer>, Constable, ConstantDesc

```
public final class Integer
extends Number
implements Comparable<Integer>, Constable, ConstantDesc
```

The `Integer` class is the wrapper class for values of the primitive type `int`. An object of type `Integer` contains a single field whose type is `int`.

In addition, this class provides several methods for converting an `int` to a `String` and a `String` to an `int`, as well as other constants and methods useful when dealing with an `int`.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail.

Implementation note: The implementations of the "bit twiddling" methods (such as `highestOneBit` and `numberOfTrailingZeros`) are based on material from Henry S. Warren, Jr.'s *Hacker's Delight*, (Addison Wesley, 2002) and *Hacker's Delight, Second Edition*, (Pearson Education, 2013).

Since:

1.0

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
`static final int`	**`BYTES`**	The number of bytes used to represent an `int` value in two's complement binary form.
`static final int`	**`MAX_VALUE`**	

		A constant holding the maximum value an `int` can have, $2^{31}-1$.
`static final int`	**`MIN_VALUE`**	A constant holding the minimum value an `int` can have, -2^{31}.
`static final int`	**`SIZE`**	The number of bits used to represent an `int` value in two's complement binary form.
`static final Class<Integer>`	**`TYPE`**	The `Class` instance representing the primitive type `int`.

Constructor Summary

Constructors

Constructor	Description
`Integer``(int value)`	**Deprecated.** It is rarely appropriate to use this constructor.
`Integer``(String s)`	**Deprecated.** It is rarely appropriate to use this constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`static int`	**`bitCount`**`(int i)`	Returns the number of one-bits in the two's complement binary representation of the specified `int` value.
`byte`	**`byteValue`**`()`	Returns the value of this `Integer` as a `byte` after a

		narrowing primitive conversion.
static int	**compare**(int x, int y)	Compares two int values numerically.
int	**compareTo**(Integer anotherInteger)	Compares two Integer objects numerically.
static int	**compareUnsigned**(int x, int y)	Compares two int values numerically treating the values as unsigned.
static int	**compress**(int i, int mask)	Returns the value obtained by compressing the bits of the specified int value, i, in accordance with the specified bit mask.
static Integer	**decode**(String nm)	Decodes a String into an Integer.
Optional<Integer>	**describeConstable**()	Returns an Optional containing the nominal descriptor for this instance, which is the instance itself.
static int	**divideUnsigned**(int dividend, int divisor)	Returns the unsigned quotient of dividing the first argument by the second where each argument and the result is interpreted as an unsigned value.
double	**doubleValue**()	Returns the value of this Integer as a double after a widening primitive conversion.
boolean	**equals**(Object obj)	

		Compares this object to the specified object.
static int	**expand**(int i, int mask)	Returns the value obtained by expanding the bits of the specified int value, i, in accordance with the specified bit mask.
float	**floatValue**()	Returns the value of this Integer as a float after a widening primitive conversion.
static Integer	**getInteger**(String nm)	Determines the integer value of the system property with the specified name.
static Integer	**getInteger**(String nm, int val)	Determines the integer value of the system property with the specified name.
static Integer	**getInteger**(String nm, Integer val)	Returns the integer value of the system property with the specified name.
int	**hashCode**()	Returns a hash code for this Integer.
static int	**hashCode**(int value)	Returns a hash code for an int value; compatible with Integer.hashCode().
static int	**highestOneBit**(int i)	Returns an int value with at most a single one-bit, in the position of the highest-order ("leftmost") one-bit in the specified int value.
int	**intValue**()	

		Returns the value of this `Integer` as an `int`.
`long`	**`longValue`**`()`	Returns the value of this `Integer` as a `long` after a widening primitive conversion.
`static int`	**`lowestOneBit`**`(int i)`	Returns an `int` value with at most a single one-bit, in the position of the lowest-order ("rightmost") one-bit in the specified `int` value.
`static int`	**`max`**`(int a, int b)`	Returns the greater of two `int` values as if by calling `Math.max`.
`static int`	**`min`**`(int a, int b)`	Returns the smaller of two `int` values as if by calling `Math.min`.
`static int`	**`numberOfLeadingZeros`**`(int i)`	Returns the number of zero bits preceding the highest-order ("leftmost") one-bit in the two's complement binary representation of the specified `int` value.
`static int`	**`numberOfTrailingZeros`**`(int i)`	Returns the number of zero bits following the lowest-order ("rightmost") one-bit in the two's complement binary representation of the specified `int` value.
`static int`	**`parseInt`**`(CharSequence s, int beginIndex, int endIndex, int radix)`	Parses the `CharSequence` argument as a signed `int` in the specified `radix`, beginning at the specified `beginIndex` and extending to `endIndex - 1`.

static int	**parseInt**(String s)	Parses the string argument as a signed decimal integer.
static int	**parseInt**(String s, int radix)	Parses the string argument as a signed integer in the radix specified by the second argument.
static int	**parseUnsignedInt**(CharSequence s, int beginIndex, int endIndex, int radix)	Parses the CharSequence argument as an unsigned int in the specified radix, beginning at the specified beginIndex and extending to endIndex - 1.
static int	**parseUnsignedInt**(String s)	Parses the string argument as an unsigned decimal integer.
static int	**parseUnsignedInt**(String s, int radix)	Parses the string argument as an unsigned integer in the radix specified by the second argument.
static int	**remainderUnsigned**(int dividend, int divisor)	Returns the unsigned remainder from dividing the first argument by the second where each argument and the result is interpreted as an unsigned value.
Integer	**resolveConstantDesc**(MethodHandles.Lookup lo	Resolves this instance as a ConstantDesc, the result of which is the instance itself.
static int	**reverse**(int i)	Returns the value obtained by reversing the order of the bits in

		the two's complement binary representation of the specified `int` value.
`static int`	**`reverseBytes`**`(int i)`	Returns the value obtained by reversing the order of the bytes in the two's complement representation of the specified `int` value.
`static int`	**`rotateLeft`**`(int i, int distance)`	Returns the value obtained by rotating the two's complement binary representation of the specified `int` value left by the specified number of bits.
`static int`	**`rotateRight`**`(int i, int distance)`	Returns the value obtained by rotating the two's complement binary representation of the specified `int` value right by the specified number of bits.
`short`	**`shortValue`**`()`	Returns the value of this `Integer` as a `short` after a narrowing primitive conversion.
`static int`	**`signum`**`(int i)`	Returns the signum function of the specified `int` value.
`static int`	**`sum`**`(int a, int b)`	Adds two integers together as per the + operator.
`static String`	**`toBinaryString`**`(int i)`	Returns a string representation of the integer argument as an

		unsigned integer in base 2.
static String	**toHexString**(int i)	Returns a string representation of the integer argument as an unsigned integer in base 16.
static String	**toOctalString**(int i)	Returns a string representation of the integer argument as an unsigned integer in base 8.
String	**toString**()	Returns a `String` object representing this `Integer`'s value.
static String	**toString**(int i)	Returns a `String` object representing the specified integer.
static String	**toString**(int i, int radix)	Returns a string representation of the first argument in the radix specified by the second argument.
static long	**toUnsignedLong**(int x)	Converts the argument to a `long` by an unsigned conversion.
static String	**toUnsignedString**(int i)	Returns a string representation of the argument as an unsigned decimal value.
static String	**toUnsignedString**(int i, int radix)	Returns a string representation of the first argument as an unsigned integer value

		in the radix specified by the second argument.
static Integer	**valueOf**(int i)	Returns an Integer instance representing the specified int value.
static Integer	**valueOf**(String s)	Returns an Integer object holding the value of the specified String.
static Integer	**valueOf**(String s, int radix)	Returns an Integer object holding the value extracted from the specified String when parsed with the radix given by the second argument.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

MIN_VALUE

```
@Native
public static final int MIN_VALUE
```

A constant holding the minimum value an int can have, -2^{31}.

See Also:

Constant Field Values

MAX_VALUE

```
@Native
public static final int MAX_VALUE
```

A constant holding the maximum value an int can have, $2^{31}-1$.

See Also:

Constant Field Values

TYPE

```
public static final Class<Integer> TYPE
```

The `Class` instance representing the primitive type `int`.

Since:

1.1

SIZE

```
@Native
public static final int SIZE
```

The number of bits used to represent an `int` value in two's complement binary form.

Since:

1.5

See Also:

Constant Field Values

BYTES

```
public static final int BYTES
```

The number of bytes used to represent an `int` value in two's complement binary form.

Since:

1.8

See Also:

Constant Field Values

Constructor Details

Integer

```
@Deprecated(since="9")
public Integer(int value)
```

Deprecated.
It is rarely appropriate to use this constructor. The static factory `valueOf(int)` is generally a better choice, as it is likely to yield significantly better space and time performance.

Constructs a newly allocated `Integer` object that represents the specified `int` value.

Parameters:

`value` - the value to be represented by the `Integer` object.

Integer

```
@Deprecated(since="9")
public Integer(String s)
        throws NumberFormatException
```

Deprecated.
It is rarely appropriate to use this constructor. Use `parseInt(String)` to convert a string to a `int` primitive, or use `valueOf(String)` to convert a string to an `Integer` object.

Constructs a newly allocated `Integer` object that represents the `int` value indicated by the `String` parameter. The string is converted to an `int` value in exactly the manner used by the `parseInt` method for radix 10.

Parameters:

`s` - the `String` to be converted to an `Integer`.

Throws:

`NumberFormatException` - if the `String` does not contain a parsable integer.

Method Details

toString

```
public static String toString(int i,
                              int radix)
```

Returns a string representation of the first argument in the radix specified by the second argument.

If the radix is smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`, then the radix `10` is used instead.

If the first argument is negative, the first element of the result is the ASCII minus character `'-'` (`'\u002D'`). If the first argument is not negative, no sign character appears in the result.

The remaining characters of the result represent the magnitude of the first argument. If the magnitude is zero, it is represented by a single zero character `'0'` (`'\u0030'`); otherwise, the first character of the representation of the magnitude will not be the zero character. The following ASCII characters are used as digits:

```
0123456789abcdefghijklmnopqrstuvwxyz
```

These are `'\u0030'` through `'\u0039'` and `'\u0061'` through `'\u007A'`. If `radix` is *N*, then the first *N* of these characters are used as radix-*N* digits in the order shown. Thus, the digits for hexadecimal (radix 16) are `0123456789abcdef`. If uppercase letters are desired, the `String.toUpperCase()` method may be called on the result:

```
Integer.toString(n, 16).toUpperCase()
```

Parameters:

`i` - an integer to be converted to a string.

`radix` - the radix to use in the string representation.

Returns:

a string representation of the argument in the specified radix.

See Also:

`Character.MAX_RADIX`, `Character.MIN_RADIX`

toUnsignedString

```
public static String toUnsignedString(int i,
                                      int radix)
```

Returns a string representation of the first argument as an unsigned integer value in the radix specified by the second argument.

If the radix is smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`, then the radix `10` is used instead.

Note that since the first argument is treated as an unsigned value, no leading sign character is printed.

If the magnitude is zero, it is represented by a single zero character `'0'` (`'\u0030'`); otherwise, the first character of the representation of the magnitude will not be the zero character.

The behavior of radixes and the characters used as digits are the same as `toString`.

Parameters:

`i` - an integer to be converted to an unsigned string.

`radix` - the radix to use in the string representation.

Returns:

an unsigned string representation of the argument in the specified radix.

Since:

1.8

See Also:

`toString(int, int)`

toHexString

```
public static String toHexString(int i)
```

Returns a string representation of the integer argument as an unsigned integer in base 16.

The unsigned integer value is the argument plus 2^{32} if the argument is negative; otherwise, it is equal to the argument. This value is converted to a string of ASCII digits in hexadecimal (base 16) with no extra leading `0`s.

The value of the argument can be recovered from the returned string `s` by calling `Integer.parseUnsignedInt(s, 16)`.

If the unsigned magnitude is zero, it is represented by a single zero character `'0'` (`'\u0030'`); otherwise, the first character of the representation of the unsigned magnitude will not be the zero character. The following characters are used as hexadecimal digits:

```
0123456789abcdef
```

These are the characters `'\u0030'` through `'\u0039'` and `'\u0061'` through `'\u0066'`. If uppercase letters are desired, the `String.toUpperCase()` method may be called on the result:

```
Integer.toHexString(n).toUpperCase()
```

API Note:

The `HexFormat` class provides formatting and parsing of byte arrays and primitives to return a string or adding to an `Appendable`. `HexFormat` formats and parses uppercase or lowercase hexadecimal characters, with leading zeros and for byte arrays includes for each byte a delimiter, prefix, and suffix.

Parameters:

`i` - an integer to be converted to a string.

Returns:

the string representation of the unsigned integer value represented by the argument in hexadecimal (base 16).

Since:

1.0.2

See Also:

`HexFormat`,
`parseUnsignedInt(String, int)`,
`toUnsignedString(int, int)`

toOctalString

```
public static String toOctalString(int i)
```

Returns a string representation of the integer argument as an unsigned integer in base 8.

The unsigned integer value is the argument plus 2^{32} if the argument is negative; otherwise, it is equal to the argument. This value is converted to a string of ASCII digits in octal (base 8) with no extra leading `0`s.

The value of the argument can be recovered from the returned string `s` by calling `Integer.parseUnsignedInt(s, 8)`.

If the unsigned magnitude is zero, it is represented by a single zero character `'0'` (`'\u0030'`); otherwise, the first character of the representation of the unsigned magnitude will not be the zero character. The following characters are used as octal digits:

```
01234567
```

These are the characters `'\u0030'` through `'\u0037'`.

Parameters:

`i` - an integer to be converted to a string.

Returns:

the string representation of the unsigned integer value represented by the argument in octal (base 8).

Since:

1.0.2

See Also:

`parseUnsignedInt(String, int)`,
`toUnsignedString(int, int)`

toBinaryString

```
public static String toBinaryString(int i)
```

Returns a string representation of the integer argument as an unsigned integer in base 2.

The unsigned integer value is the argument plus 2^{32} if the argument is negative; otherwise it is equal to the argument. This value is converted to a string of ASCII digits in binary (base 2) with no extra leading `0`s.

The value of the argument can be recovered from the returned string `s` by calling `Integer.parseUnsignedInt(s, 2)`.

If the unsigned magnitude is zero, it is represented by a single zero character `'0'` (`'\u0030'`); otherwise, the first character of the representation of the unsigned magnitude will not be the zero character. The characters `'0'` (`'\u0030'`) and `'1'` (`'\u0031'`) are used as binary digits.

Parameters:

`i` - an integer to be converted to a string.

Returns:

the string representation of the unsigned integer value represented by the argument in binary (base 2).

Since:

1.0.2

See Also:

`parseUnsignedInt(String, int)`,
`toUnsignedString(int, int)`

toString

```
public static String toString(int i)
```

Returns a `String` object representing the specified integer. The argument is converted to signed decimal representation and returned as a string, exactly as if the argument and radix 10 were given as arguments to the `toString(int, int)` method.

Parameters:

`i` - an integer to be converted.

Returns:

a string representation of the argument in base 10.

toUnsignedString

```
public static String toUnsignedString(int i)
```

Returns a string representation of the argument as an unsigned decimal value. The argument is converted to unsigned decimal representation and returned as a string exactly as if the argument and radix 10 were given as arguments to the `toUnsignedString(int, int)` method.

Parameters:

`i` - an integer to be converted to an unsigned string.

Returns:

an unsigned string representation of the argument.

Since:

1.8

See Also:

`toUnsignedString(int, int)`

parseInt

```
public static int parseInt(String s,
                           int radix)
                    throws NumberFormatException
```

Parses the string argument as a signed integer in the radix specified by the second argument. The characters in the string must all be digits of the specified radix (as determined by whether `Character.digit(char, int)` returns a nonnegative value), except that the first character may be an ASCII minus sign `'-'` (`'\u002D'`) to indicate a negative value or an ASCII plus sign `'+'` (`'\u002B'`) to indicate a positive value. The resulting integer value is returned.

An exception of type `NumberFormatException` is thrown if any of the following situations occurs:

- The first argument is `null` or is a string of length zero.
- The radix is either smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`.
- Any character of the string is not a digit of the specified radix, except that the first character may be a minus sign `'-'` (`'\u002D'`) or plus sign `'+'` (`'\u002B'`) provided that the string is longer than length 1.
- The value represented by the string is not a value of type `int`.

Examples:

```
parseInt("0", 10) returns 0
parseInt("473", 10) returns 473
parseInt("+42", 10) returns 42
parseInt("-0", 10) returns 0
parseInt("-FF", 16) returns -255
parseInt("1100110", 2) returns 102
parseInt("2147483647", 10) returns 2147483647
parseInt("-2147483648", 10) returns -2147483648
parseInt("2147483648", 10) throws a NumberFormatException
parseInt("99", 8) throws a NumberFormatException
parseInt("Kona", 10) throws a NumberFormatException
parseInt("Kona", 27) returns 411787
```

Parameters:

`s` - the `String` containing the integer representation to be parsed

`radix` - the radix to be used while parsing `s`.

Returns:

the integer represented by the string argument in the specified radix.

Throws:

`NumberFormatException` - if the `String` does not contain a parsable `int`.

parseInt

```
public static int parseInt(CharSequence s,
                           int beginIndex,
                           int endIndex,
                           int radix)
                    throws NumberFormatException
```

Parses the `CharSequence` argument as a signed `int` in the specified `radix`, beginning at the specified `beginIndex` and extending to `endIndex - 1`.

The method does not take steps to guard against the `CharSequence` being mutated while parsing.

Parameters:

`s` - the `CharSequence` containing the `int` representation to be parsed

`beginIndex` - the beginning index, inclusive.

`endIndex` - the ending index, exclusive.

`radix` - the radix to be used while parsing `s`.

Returns:

the signed `int` represented by the subsequence in the specified radix.

Throws:

`NullPointerException` - if `s` is null.

`IndexOutOfBoundsException` - if `beginIndex` is negative, or if `beginIndex` is greater than `endIndex` or if `endIndex` is greater than `s.length()`.

`NumberFormatException` - if the `CharSequence` does not contain a parsable `int` in the specified `radix`, or if `radix` is either smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`.

Since:

9

parseInt

```
public static int parseInt(String s)
                    throws NumberFormatException
```

Parses the string argument as a signed decimal integer. The characters in the string must all be decimal digits, except that the first character may be an ASCII minus sign `'-'` (`'\u002D'`) to indicate a negative value or an ASCII plus sign `'+'` (`'\u002B'`) to indicate a positive value. The resulting integer value is returned, exactly as if the argument and the radix 10 were given as arguments to the `parseInt(java.lang.String, int)` method.

Parameters:

`s` - a `String` containing the `int` representation to be parsed

Returns:

the integer value represented by the argument in decimal.

Throws:

`NumberFormatException` - if the string does not contain a parsable integer.

parseUnsignedInt

```
public static int parseUnsignedInt(String s,
                                   int radix)
                            throws NumberFormatException
```

Parses the string argument as an unsigned integer in the radix specified by the second argument. An unsigned integer maps the values usually associated with negative numbers to positive numbers larger than `MAX_VALUE`. The characters in the string must all be digits of the specified radix (as determined by whether `Character.digit(char, int)` returns a nonnegative value), except that the first character may be an ASCII plus sign `'+'` (`'\u002B'`). The resulting integer value is returned.

An exception of type `NumberFormatException` is thrown if any of the following situations occurs:

- The first argument is `null` or is a string of length zero.
- The radix is either smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`.
- Any character of the string is not a digit of the specified radix, except that the first character may be a plus sign `'+'` (`'\u002B'`) provided that the string is longer than length 1.
- The value represented by the string is larger than the largest unsigned `int`, 2^{32}-1.

Parameters:

`s` - the `String` containing the unsigned integer representation to be parsed

`radix` - the radix to be used while parsing `s`.

Returns:

the integer represented by the string argument in the specified radix.

Throws:

`NumberFormatException` - if the `String` does not contain a parsable `int`.

Since:

1.8

parseUnsignedInt

```
public static int parseUnsignedInt(CharSequence s,
                                   int beginIndex,
                                   int endIndex,
                                   int radix)
                            throws NumberFormatException
```

Parses the `CharSequence` argument as an unsigned `int` in the specified `radix`, beginning at the specified `beginIndex` and extending to `endIndex - 1`.

The method does not take steps to guard against the `CharSequence` being mutated while parsing.

Parameters:

`s` - the `CharSequence` containing the unsigned `int` representation to be parsed

`beginIndex` - the beginning index, inclusive.

`endIndex` - the ending index, exclusive.

`radix` - the radix to be used while parsing `s`.

Returns:

the unsigned `int` represented by the subsequence in the specified radix.

Throws:

`NullPointerException` - if `s` is null.

`IndexOutOfBoundsException` - if `beginIndex` is negative, or if `beginIndex` is greater than `endIndex` or if `endIndex` is greater than `s.length()`.

`NumberFormatException` - if the `CharSequence` does not contain a parsable unsigned `int` in the specified `radix`, or if `radix` is either smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`.

Since:

9

parseUnsignedInt

```
public static int parseUnsignedInt(String s)
                            throws NumberFormatException
```

Parses the string argument as an unsigned decimal integer. The characters in the string must all be decimal digits, except that the first character may be an ASCII plus sign `'+'` (`'\u002B'`). The resulting integer value is returned, exactly as if the argument and the radix 10 were given as arguments to the `parseUnsignedInt(java.lang.String, int)` method.

Parameters:

`s` - a `String` containing the unsigned `int` representation to be parsed

Returns:

the unsigned integer value represented by the argument in decimal.

Throws:

`NumberFormatException` - if the string does not contain a parsable unsigned integer.

Since:

1.8

valueOf

```
public static Integer valueOf(String s,
                              int radix)
                       throws NumberFormatException
```

Returns an `Integer` object holding the value extracted from the specified `String` when parsed with the radix given by the second argument. The first argument is interpreted as representing a signed integer in the radix specified by the second argument, exactly as if the arguments were given to

the parseInt(java.lang.String, int) method. The result is an Integer object that represents the integer value specified by the string.

In other words, this method returns an Integer object equal to the value of:

```
Integer.valueOf(Integer.parseInt(s, radix))
```

Parameters:

s - the string to be parsed.

radix - the radix to be used in interpreting s

Returns:

an Integer object holding the value represented by the string argument in the specified radix.

Throws:

NumberFormatException - if the String does not contain a parsable int.

valueOf

```
public static Integer valueOf(String s)
                       throws NumberFormatException
```

Returns an Integer object holding the value of the specified String. The argument is interpreted as representing a signed decimal integer, exactly as if the argument were given to the parseInt(java.lang.String) method. The result is an Integer object that represents the integer value specified by the string.

In other words, this method returns an Integer object equal to the value of:

```
Integer.valueOf(Integer.parseInt(s))
```

Parameters:

s - the string to be parsed.

Returns:

an Integer object holding the value represented by the string argument.

Throws:

NumberFormatException - if the string cannot be parsed as an integer.

valueOf

```
public static Integer valueOf(int i)
```

Returns an Integer instance representing the specified int value. If a new Integer instance is not required, this method should generally be used in preference to the constructor Integer(int), as this method is likely to yield

significantly better space and time performance by caching frequently requested values. This method will always cache values in the range -128 to 127, inclusive, and may cache other values outside of this range.

Parameters:

`i` - an `int` value.

Returns:

an `Integer` instance representing `i`.

Since:

1.5

byteValue

```
public byte byteValue()
```

Returns the value of this `Integer` as a `byte` after a narrowing primitive conversion.

Overrides:

`byteValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `byte`.

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

shortValue

```
public short shortValue()
```

Returns the value of this `Integer` as a `short` after a narrowing primitive conversion.

Overrides:

`shortValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `short`.

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

intValue

```
public int intValue()
```

Returns the value of this `Integer` as an `int`.

Specified by:

`intValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `int`.

longValue

```
public long longValue()
```

Returns the value of this `Integer` as a `long` after a widening primitive conversion.

Specified by:

`longValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `long`.

See *Java Language Specification*:

5.1.2 Widening Primitive Conversion

See Also:

`toUnsignedLong(int)`

floatValue

```
public float floatValue()
```

Returns the value of this `Integer` as a `float` after a widening primitive conversion.

Specified by:

`floatValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `float`.

See *Java Language Specification*:

5.1.2 Widening Primitive Conversion

doubleValue

```
public double doubleValue()
```

Returns the value of this `Integer` as a `double` after a widening primitive conversion.

Specified by:

`doubleValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `double`.

See *Java Language Specification*:

5.1.2 Widening Primitive Conversion

toString

```
public String toString()
```

Returns a `String` object representing this `Integer`'s value. The value is converted to signed decimal representation and returned as a string, exactly as if the integer value were given as an argument to the `toString(int)` method.

Overrides:

`toString` in class `Object`

Returns:

a string representation of the value of this object in base 10.

hashCode

```
public int hashCode()
```

Returns a hash code for this `Integer`.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this object, equal to the primitive `int` value represented by this `Integer` object.

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(Object)`

hashCode

```
public static int hashCode(int value)
```

Returns a hash code for an `int` value; compatible with `Integer.hashCode()`.

Parameters:

`value` - the value to hash

Returns:

a hash code value for an `int` value.

Since:

1.8

equals

```
public boolean equals(Object obj)
```

Compares this object to the specified object. The result is `true` if and only if the argument is not `null` and is an `Integer` object that contains the same `int` value as this object.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to compare with.

Returns:

`true` if the objects are the same; `false` otherwise.

See Also:

`Object.hashCode()`, `HashMap`

getInteger

```
public static Integer getInteger(String nm)
```

Determines the integer value of the system property with the specified name.

The first argument is treated as the name of a system property. System properties are accessible through the `System.getProperty(java.lang.String)` method. The string value of this property is then interpreted as an integer value using the grammar supported by `decode` and an `Integer` object representing this value is returned.

If there is no property with the specified name, if the specified name is empty or `null`, or if the property does not have the correct numeric format, then `null` is returned.

In other words, this method returns an `Integer` object equal to the value of:

```
getInteger(nm, null)
```

Parameters:

`nm` - property name.

Returns:

the `Integer` value of the property.

See Also:

System.getProperty(java.lang.String),
System.getProperty(java.lang.String, java.lang.String)

getInteger

```
public static Integer getInteger(String nm,
                                 int val)
```

Determines the integer value of the system property with the specified name.

The first argument is treated as the name of a system property. System properties are accessible through the System.getProperty(java.lang.String) method. The string value of this property is then interpreted as an integer value using the grammar supported by decode and an Integer object representing this value is returned.

The second argument is the default value. An Integer object that represents the value of the second argument is returned if there is no property of the specified name, if the property does not have the correct numeric format, or if the specified name is empty or null.

In other words, this method returns an Integer object equal to the value of:

```
getInteger(nm, Integer.valueOf(val))
```

but in practice it may be implemented in a manner such as:

```
Integer result = getInteger(nm, null);
return (result == null) ? Integer.valueOf(val) : result;
```

to avoid the unnecessary allocation of an Integer object when the default value is not needed.

Parameters:

nm - property name.

val - default value.

Returns:

the Integer value of the property.

See Also:

System.getProperty(java.lang.String),
System.getProperty(java.lang.String, java.lang.String)

getInteger

```
public static Integer getInteger(String nm,
                                 Integer val)
```

Returns the integer value of the system property with the specified name. The first argument is treated as the name of a system property. System properties are accessible through the `System.getProperty(java.lang.String)` method. The string value of this property is then interpreted as an integer value, as per the `decode` method, and an `Integer` object representing this value is returned; in summary:

- If the property value begins with the two ASCII characters `0x` or the ASCII character `#`, not followed by a minus sign, then the rest of it is parsed as a hexadecimal integer exactly as by the method `valueOf(java.lang.String, int)` with radix 16.
- If the property value begins with the ASCII character `0` followed by another character, it is parsed as an octal integer exactly as by the method `valueOf(java.lang.String, int)` with radix 8.
- Otherwise, the property value is parsed as a decimal integer exactly as by the method `valueOf(java.lang.String, int)` with radix 10.

The second argument is the default value. The default value is returned if there is no property of the specified name, if the property does not have the correct numeric format, or if the specified name is empty or `null`.

Parameters:

`nm` - property name.

`val` - default value.

Returns:

the `Integer` value of the property.

See Also:

`System.getProperty(java.lang.String)`,
`System.getProperty(java.lang.String, java.lang.String)`

decode

```
public static Integer decode(String nm)
                     throws NumberFormatException
```

Decodes a `String` into an `Integer`. Accepts decimal, hexadecimal, and octal numbers given by the following grammar:

DecodableString:
$Sign_{opt}$ *DecimalNumeral*

Sign$_{opt}$ `0x` *HexDigits*

Sign$_{opt}$ `0X` *HexDigits*

Sign$_{opt}$ `#` *HexDigits*

Sign$_{opt}$ `0` *OctalDigits*

Sign:

-

+

DecimalNumeral, *HexDigits*, and *OctalDigits* are as defined in section §3.10.1 of *The Java Language Specification*, except that underscores are not accepted between digits.

The sequence of characters following an optional sign and/or radix specifier ("`0x`", "`0X`", "`#`", or leading zero) is parsed as by the `Integer.parseInt` method with the indicated radix (10, 16, or 8). This sequence of characters must represent a positive value or a `NumberFormatException` will be thrown. The result is negated if first character of the specified `String` is the minus sign. No whitespace characters are permitted in the `String`.

Parameters:

`nm` - the `String` to decode.

Returns:

an `Integer` object holding the `int` value represented by `nm`

Throws:

`NumberFormatException` - if the `String` does not contain a parsable integer.

See Also:

`parseInt(java.lang.String, int)`

compareTo

```
public int compareTo(Integer anotherInteger)
```

Compares two `Integer` objects numerically.

Specified by:

`compareTo` in interface `Comparable<Integer>`

Parameters:

`anotherInteger` - the `Integer` to be compared.

Returns:

the value `0` if this `Integer` is equal to the argument `Integer`; a value less than `0` if this `Integer` is numerically less than the argument `Integer`; and a value greater than `0` if this `Integer` is numerically greater than the argument `Integer` (signed comparison).

Since:

1.2

compare

```
public static int compare(int x,
                          int y)
```

Compares two `int` values numerically. The value returned is identical to what would be returned by:

```
   Integer.valueOf(x).compareTo(Integer.valueOf(y))
```

Parameters:

`x` - the first `int` to compare

`y` - the second `int` to compare

Returns:

the value `0` if `x == y`; a value less than `0` if `x < y`; and a value greater than `0` if `x > y`

Since:

1.7

compareUnsigned

```
public static int compareUnsigned(int x,
                                  int y)
```

Compares two `int` values numerically treating the values as unsigned.

Parameters:

`x` - the first `int` to compare

`y` - the second `int` to compare

Returns:

the value `0` if `x == y`; a value less than `0` if `x < y` as unsigned values; and a value greater than `0` if `x > y` as unsigned values

Since:

1.8

toUnsignedLong

```
public static long toUnsignedLong(int x)
```

Converts the argument to a `long` by an unsigned conversion. In an unsigned conversion to a `long`, the high-order 32 bits of the `long` are zero and the low-

order 32 bits are equal to the bits of the integer argument. Consequently, zero and positive `int` values are mapped to a numerically equal `long` value and negative `int` values are mapped to a `long` value equal to the input plus 2^{32}.

Parameters:

`x` - the value to convert to an unsigned `long`

Returns:

the argument converted to `long` by an unsigned conversion

Since:

1.8

divideUnsigned

```
public static int divideUnsigned(int dividend,
                                 int divisor)
```

Returns the unsigned quotient of dividing the first argument by the second where each argument and the result is interpreted as an unsigned value.

Note that in two's complement arithmetic, the three other basic arithmetic operations of add, subtract, and multiply are bit-wise identical if the two operands are regarded as both being signed or both being unsigned. Therefore separate `addUnsigned`, etc. methods are not provided.

Parameters:

`dividend` - the value to be divided

`divisor` - the value doing the dividing

Returns:

the unsigned quotient of the first argument divided by the second argument

Throws:

`ArithmeticException` - if the divisor is zero

Since:

1.8

See Also:

`remainderUnsigned(int, int)`

remainderUnsigned

```
public static int remainderUnsigned(int dividend,
                                    int divisor)
```

Returns the unsigned remainder from dividing the first argument by the second where each argument and the result is interpreted as an unsigned value.

Parameters:

`dividend` - the value to be divided

`divisor` - the value doing the dividing

Returns:

the unsigned remainder of the first argument divided by the second argument

Throws:

`ArithmeticException` - if the divisor is zero

Since:

1.8

See Also:

`divideUnsigned(int, int)`

highestOneBit

```
public static int highestOneBit(int i)
```

Returns an `int` value with at most a single one-bit, in the position of the highest-order ("leftmost") one-bit in the specified `int` value. Returns zero if the specified value has no one-bits in its two's complement binary representation, that is, if it is equal to zero.

Parameters:

`i` - the value whose highest one bit is to be computed

Returns:

an `int` value with a single one-bit, in the position of the highest-order one-bit in the specified value, or zero if the specified value is itself equal to zero.

Since:

1.5

lowestOneBit

```
public static int lowestOneBit(int i)
```

Returns an `int` value with at most a single one-bit, in the position of the lowest-order ("rightmost") one-bit in the specified `int` value. Returns zero if the specified value has no one-bits in its two's complement binary representation, that is, if it is equal to zero.

Parameters:

`i` - the value whose lowest one bit is to be computed

Returns:

an `int` value with a single one-bit, in the position of the lowest-order one-bit in the specified value, or zero if the specified value is itself equal to zero.

Since:

1.5

numberOfLeadingZeros

```
public static int numberOfLeadingZeros(int i)
```

Returns the number of zero bits preceding the highest-order ("leftmost") one-bit in the two's complement binary representation of the specified `int` value. Returns 32 if the specified value has no one-bits in its two's complement representation, in other words if it is equal to zero.

Note that this method is closely related to the logarithm base 2. For all positive `int` values x:

- floor($\log_2$(x)) = `31 - numberOfLeadingZeros(x)`
- ceil($\log_2$(x)) = `32 - numberOfLeadingZeros(x - 1)`

Parameters:

`i` - the value whose number of leading zeros is to be computed

Returns:

the number of zero bits preceding the highest-order ("leftmost") one-bit in the two's complement binary representation of the specified `int` value, or 32 if the value is equal to zero.

Since:

1.5

numberOfTrailingZeros

```
public static int numberOfTrailingZeros(int i)
```

Returns the number of zero bits following the lowest-order ("rightmost") one-bit in the two's complement binary representation of the specified `int` value. Returns 32 if the specified value has no one-bits in its two's complement representation, in other words if it is equal to zero.

Parameters:

`i` - the value whose number of trailing zeros is to be computed

Returns:

the number of zero bits following the lowest-order ("rightmost") one-bit in the two's complement binary representation of the specified `int` value, or 32 if the value is equal to zero.

Since:

1.5

bitCount

```
public static int bitCount(int i)
```

Returns the number of one-bits in the two's complement binary representation of the specified `int` value. This function is sometimes referred to as the *population count*.

Parameters:

`i` - the value whose bits are to be counted

Returns:

the number of one-bits in the two's complement binary representation of the specified `int` value.

Since:

1.5

rotateLeft

```
public static int rotateLeft(int i,
                             int distance)
```

Returns the value obtained by rotating the two's complement binary representation of the specified `int` value left by the specified number of bits. (Bits shifted out of the left hand, or high-order, side reenter on the right, or low-order.)

Note that left rotation with a negative distance is equivalent to right rotation: `rotateLeft(val, -distance) == rotateRight(val, distance)`. Note also that rotation by any multiple of 32 is a no-op, so all but the last five bits of the rotation distance can be ignored, even if the distance is negative: `rotateLeft(val, distance) == rotateLeft(val, distance & 0x1F)`.

Parameters:

`i` - the value whose bits are to be rotated left

`distance` - the number of bit positions to rotate left

Returns:

the value obtained by rotating the two's complement binary representation of the specified `int` value left by the specified number of bits.

Since:

1.5

rotateRight

```
public static int rotateRight(int i,
                              int distance)
```

Returns the value obtained by rotating the two's complement binary representation of the specified `int` value right by the specified number of bits. (Bits shifted out of the right hand, or low-order, side reenter on the left, or high-order.)

Note that right rotation with a negative distance is equivalent to left rotation: `rotateRight(val, -distance) == rotateLeft(val, distance)`. Note also that rotation by any multiple of 32 is a no-op, so all but the last five bits of the rotation distance can be ignored, even if the distance is negative: `rotateRight(val, distance) == rotateRight(val, distance & 0x1F)`.

Parameters:

`i` - the value whose bits are to be rotated right

`distance` - the number of bit positions to rotate right

Returns:

the value obtained by rotating the two's complement binary representation of the specified `int` value right by the specified number of bits.

Since:

1.5

reverse

```
public static int reverse(int i)
```

Returns the value obtained by reversing the order of the bits in the two's complement binary representation of the specified `int` value.

Parameters:

`i` - the value to be reversed

Returns:

the value obtained by reversing order of the bits in the specified `int` value.

Since:

1.5

compress

```
public static int compress(int i,
                           int mask)
```

Returns the value obtained by compressing the bits of the specified `int` value, `i`, in accordance with the specified bit mask.

For each one-bit value `mb` of the mask, from least significant to most significant, the bit value of `i` at the same bit location as `mb` is assigned to the compressed value contiguously starting from the least significant bit location. All the upper remaining bits of the compressed value are set to zero.

API Note:

Consider the simple case of compressing the digits of a hexadecimal value:

```
// Compressing drink to food
compress(0xCAFEBABE, 0xFF00FFF0) == 0xCABAB
```

Starting from the least significant hexadecimal digit at position 0 from the right, the mask `0xFF00FFF0` selects hexadecimal digits at positions 1, 2, 3, 6 and 7 of `0xCAFEBABE`. The selected digits occur in the resulting compressed value contiguously from digit position 0 in the same order.

The following identities all return `true` and are helpful to understand the behaviour of `compress`:

```
// Returns 1 if the bit at position n is one
compress(x, 1 << n) == (x >> n & 1)

// Logical shift right
compress(x, -1 << n) == x >>> n

// Any bits not covered by the mask are ignored
compress(x, m) == compress(x & m, m)

// Compressing a value by itself
compress(m, m) == (m == -1 || m == 0) ? m : (1 << bitCount(m)) - 1

// Expanding then compressing with the same mask
compress(expand(x, m), m) == x & compress(m, m)
```

The Sheep And Goats (SAG) operation (see Hacker's Delight, Second Edition, section 7.7) can be implemented as follows:

```
int compressLeft(int i, int mask) {
    // This implementation follows the description in Hacker's Delight which
    // is informative. A more optimal implementation is:
    //   Integer.compress(i, mask) << -Integer.bitCount(mask)
    return Integer.reverse(
        Integer.compress(Integer.reverse(i), Integer.reverse(mask)));
}

int sag(int i, int mask) {
    return compressLeft(i, mask) | Integer.compress(i, ~mask);
}

// Separate the sheep from the goats
sag(0xCAFEBABE, 0xFF00FFF0) == 0xCABABFEE
```

Parameters:

`i` - the value whose bits are to be compressed

`mask` - the bit mask

Returns:

the compressed value

Since:

19

See Also:

`expand(int, int)`

expand

```
public static int expand(int i,
                         int mask)
```

Returns the value obtained by expanding the bits of the specified `int` value, `i`, in accordance with the specified bit mask.

For each one-bit value `mb` of the mask, from least significant to most significant, the next contiguous bit value of `i` starting at the least significant bit is assigned to the expanded value at the same bit location as `mb`. All other remaining bits of the expanded value are set to zero.

API Note:

Consider the simple case of expanding the digits of a hexadecimal value:

```
expand(0x0000CABAB, 0xFF00FFF0) == 0xCA00BAB0
```

Starting from the least significant hexadecimal digit at position 0 from the right, the mask `0xFF00FFF0` selects the first five hexadecimal digits of `0x0000CABAB`. The selected digits occur in the resulting expanded value in order at positions 1, 2, 3, 6, and 7.

The following identities all return `true` and are helpful to understand the behaviour of `expand`:

```
// Logically shift right the bit at position 0
expand(x, 1 << n) == (x & 1) << n

// Logically shift right
expand(x, -1 << n) == x << n

// Expanding all bits returns the mask
expand(-1, m) == m

// Any bits not covered by the mask are ignored
expand(x, m) == expand(x, m) & m

// Compressing then expanding with the same mask
expand(compress(x, m), m) == x & m
```

The select operation for determining the position of the one-bit with index `n` in a `int` value can be implemented as follows:

```
int select(int i, int n) {
    // the one-bit in i (the mask) with index n
    int nthBit = Integer.expand(1 << n, i);
    // the bit position of the one-bit with index n
    return Integer.numberOfTrailingZeros(nthBit);
}

// The one-bit with index 0 is at bit position 1
select(0b10101010_10101010, 0) == 1
// The one-bit with index 3 is at bit position 7
select(0b10101010_10101010, 3) == 7
```

Parameters:

`i` - the value whose bits are to be expanded

`mask` - the bit mask

Returns:

the expanded value

Since:

19

See Also:

`compress(int, int)`

signum

```
public static int signum(int i)
```

Returns the signum function of the specified `int` value. (The return value is -1 if the specified value is negative; 0 if the specified value is zero; and 1 if the specified value is positive.)

Parameters:

`i` - the value whose signum is to be computed

Returns:

the signum function of the specified `int` value.

Since:

1.5

reverseBytes

```
public static int reverseBytes(int i)
```

Returns the value obtained by reversing the order of the bytes in the two's complement representation of the specified `int` value.

Parameters:

`i` - the value whose bytes are to be reversed

Returns:

the value obtained by reversing the bytes in the specified `int` value.

Since:

1.5

sum

```
public static int sum(int a,
                      int b)
```

Adds two integers together as per the + operator.

Parameters:

`a` - the first operand

b - the second operand

Returns:

the sum of a and b

Since:

1.8

See Also:

BinaryOperator

max

```
public static int max(int a,
                      int b)
```

Returns the greater of two int values as if by calling Math.max.

Parameters:

a - the first operand

b - the second operand

Returns:

the greater of a and b

Since:

1.8

See Also:

BinaryOperator

min

```
public static int min(int a,
                      int b)
```

Returns the smaller of two int values as if by calling Math.min.

Parameters:

a - the first operand

b - the second operand

Returns:

the smaller of a and b

Since:

1.8

See Also:

BinaryOperator

describeConstable

```
public Optional<Integer> describeConstable()
```

Returns an `Optional` containing the nominal descriptor for this instance, which is the instance itself.

Specified by:

`describeConstable` in interface `Constable`

Returns:

an `Optional` describing the `Integer` instance

Since:

12

resolveConstantDesc

```
public Integer resolveConstantDesc(MethodHandles.Lookup lookup)
```

Resolves this instance as a `ConstantDesc`, the result of which is the instance itself.

Specified by:

`resolveConstantDesc` in interface `ConstantDesc`

Parameters:

`lookup` - ignored

Returns:

the `Integer` instance

Since:

12

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.